



Erlang Webmachine

Our previous article in this series, 'Exploring Software—Webservice using Mochiweb', dealt with building a simple Web application. In this article, let us explore Erlang Webmachine.

In a Web application, the browser sends a request, and the server calls a function and returns a response. Each request is independent. This maps very well to the functional programming paradigm; hence, we continue our experiments with Erlang.

Webmachine is a REST toolkit in Erlang on top of Mochiweb. As an illustration, let's start with the demo application included with Webmachine and add a form to it. The form will allow us to add a key/value pair or get all the key/value pairs that have some value.

Getting started

Let us use the packages included in the Fedora 20 repositories.

Make a copy of the demo in `/usr/share/doc/erlang-webmachine/demo/`. As the demo depends on an earlier version of Webmachine, some extra steps may be needed. In the demo directory, do the following:

```
$ rm rebar.config
$ rebar compile
Note: install additional modules in case of dependency issues
$ ./start.sh
```

The application failed to start with messages, which include phrases like 'not_started,ssl'. To overcome the errors, the start function in `src/webmachine_demo.erl` was modified as follows:

```
start() ->
    ensure_started(inets),
    ensure_started(crypto),
    ensure_started(syntax_tools),
    ensure_started(compiler),
    ensure_started(xmerl),
    ensure_started(asn1),
    ensure_started(public_key),
    ensure_started(ssl),
    ensure_started(mochiweb),
    application:set_env(webmachine, webmachine_logger_module,
                        webmachine_logger),
    ensure_started(webmachine),
```

```
application:start(webmachine_demo).
```

Now, the compilation and execution of `start.sh` should go through properly. Browse `http://localhost:8000/demo` and you should get a welcome message.

Adding functionality

The mapping between the URL paths and the modules is specified in file `priv/dispatch.conf`. Let us suppose the additional functionality is to be accessed from the URL path `/key`. Then the `priv/dispatch.conf` will become:

```
%%*- mode: erlang -*-
[{"demo", ""}, webmachine_demo_resource, []].
[{"key", ""}, webmachine_key_resource, []].
[{"fs", ""}, webmachine_demo_fs_resource, [{"root", "/tmp/fs"}]].
```

The code needed for this functionality will be in `src/webmachine_key_resource.erl`:

```
-module(webmachine_key_resource).
-export([init/1, to_html/2]).
-include_lib("webmachine/include/webmachine.hrl").
init([]) -> Context = undefined,
    {ok, Context }.

to_html(ReqData, Context) ->
    % if the key is not null, store the key/value pair in
    mnesia table
    case wrq:get_qs_value("key", ReqData) of
        Key when Key==undefined, Key="" -> ok;
        Key ->
            db:insert_key(Key, wrq:get_qs_value("value", ReqData))
    end,
    % get all the key/value pairs which match the value in
    search_for
    {Result, SendList} = db:match_value(wrq:get_qs_value("search_for", ReqData)),
    % return the result in webapp template
```